

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ

BÁO CÁO BÀI TẬP LỚN

Lập trình ứng dụng di động



ROOMHUB – ỨNG DỤNG ĐẶT PHÒNG HỌC, PHÒNG HỌP

Nhóm thực hiện:

Nguyễn Việt Dũng MSSV: 20252690M

Nguyễn Đình Phúc MSSV: 20251012M

Đỗ Thùy Linh MSSV: 20252478M

Hoàng Văn Tùng MSSV: 20252389M

Giảng viên hướng dẫn: PGS.TS. Đỗ Trọng Tuấn

Mã nguồn: <https://github.com/xwomen1/classroom-booking>

HÀ NỘI, 10/2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi	1
1.3 Định hướng giải pháp và tổ chức báo cáo	1
CHƯƠNG 2. TRIỂN KHAI HỆ THỐNG	2
2.1 Tổng quan hệ thống.....	2
2.2 Phạm vi MVP.....	3
2.3 Tổ chức mã nguồn ứng dụng	4
2.4 Lập trình theo từng module.....	4
2.4.1 Module auth và account_management.....	5
2.4.2 Module room_management.....	5
2.4.3 Module booking.....	6
2.4.4 Module schedule_maintenance.....	7
2.4.5 Module access_control	7
2.4.6 Module smart_lock và tích hợp OneIoT.....	7
2.4.7 Module notifications, profile và configuration	8
2.4.8 Booking server	8
2.5 Kết quả giao diện.....	8
2.6 Phân công và đóng góp	9
CHƯƠNG 3. QUY TRÌNH VÀ KỊCH BẢN KIỂM THỬ.....	11
3.1 Mục tiêu và phương pháp kiểm thử.....	11
3.2 Môi trường kiểm thử trên thiết bị.....	11
3.3 Quy trình kiểm thử trên thiết bị.....	11
3.4 Kịch bản kiểm thử trên thiết bị	12
3.5 Kết quả kiểm thử và lỗi đã phát hiện.....	14
KẾT LUẬN	17
TÀI LIỆU THAM KHẢO.....	19
PHỤ LỤC.....	21

A. TỔNG THỂ THÀNH PHẦN DỰ ÁN	21
A.1 Cấu trúc kho mã nguồn	21
A.2 Build và cài đặt.....	21
A.3 Tổng hợp tài liệu và phụ lục	22
B. PROMPT SỬ DỤNG TRONG QUÁ TRÌNH PHÁT TRIỂN	23
B.1 Nguyên tắc viết prompt.....	23

DANH MỤC HÌNH VẼ

Hình 2.1	Kiến trúc tổng thể của hệ thống RoomHub	2
Hình 2.2	Vòng đời trạng thái của một booking	4
Hình 2.3	Giao diện phía người dùng	9
Hình 2.4	Giao diện phía quản trị viên	9
Hình 3.1	Quy trình kiểm thử trên thiết bị thực tế	11

DANH MỤC BẢNG BIỂU

Bảng 2.1	Các thành phần chính của hệ thống RoomHub	2
Bảng 2.2	Phạm vi MVP đã chốt	3
Bảng 2.3	Danh sách module của ứng dụng	5
Bảng 2.4	Phân công công việc trong nhóm	10
Bảng 3.1	Môi trường kiểm thử trên thiết bị thực tế	11
Bảng 3.2	Kịch bản kiểm thử đầu cuối trên thiết bị thực tế	12
Bảng 3.3	Kết quả thực hiện kịch bản trên thiết bị	15
Bảng 3.4	Danh sách lỗi và vấn đề đã báo cho nhóm	15
Bảng A.1	Tài liệu đi kèm dự án	22

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
API	Giao diện lập trình ứng dụng (Application Programming Interface)
APK	Gói cài đặt ứng dụng Android (Android Package)
Booking	Một yêu cầu đặt phòng gồm phòng, ngày, khung giờ và mục đích
CRUD	Tạo, đọc, cập nhật, xóa (Create, Read, Update, Delete)
IoT	Internet vạn vật (Internet of Things)
MQTT	Giao thức truyền thông điệp publish/subscribe (Message Queuing Telemetry Transport)
MVP	Sản phẩm khả dụng tối thiểu (Minimum Viable Product)
No-show	Người đặt không có mặt sử dụng phòng đã được duyệt
PIN	Mã số dùng để mở khóa (Personal Identification Number)
TLS	Giao thức bảo mật tầng vận chuyển (Transport Layer Security)
UAT	Kiểm thử chấp nhận người dùng (User Acceptance Testing)

CHƯƠNG 1. GIỚI THIỆU

1.1 Đặt vấn đề

Nhu cầu mượn phòng học và phòng họp của giảng viên diễn ra thường xuyên, nhưng việc mượn phòng hiện nay vẫn chủ yếu dựa trên trao đổi trực tiếp với cán bộ quản lý cơ sở vật chất. Người mượn khó biết phòng nào còn trống trong khung giờ mong muốn, thiếu thông tin nhanh về sức chứa, thiết bị và vị trí phòng trên từng tầng của tòa nhà tám tầng. Về phía quản lý, việc theo dõi lịch sử dụng, lịch bảo trì, bàn giao chìa khóa hoặc thẻ từ và xử lý trường hợp người đặt không đến được thực hiện thủ công, dễ dẫn đến trùng lịch và lãng phí phòng.

Bên cạnh đó, một số phòng đã được trang bị khóa mã số có thể quản lý trực tuyến. Nếu hệ thống đặt phòng cấp được mật khẩu tạm thời cho khóa đúng theo khung giờ đã duyệt, người mượn có thể vào phòng mà không cần nhận chìa khóa, đồng thời quản lý có bằng chứng check-in và check-out.

1.2 Mục tiêu và phạm vi

Mục tiêu của dự án là xây dựng ứng dụng di động **RoomHub** quản lý xuyên suốt vòng đời một lượt mượn phòng, từ tìm phòng, đặt phòng, phê duyệt, cấp quyền truy cập đến sử dụng và kết thúc phiên. Hệ thống phục vụ hai nhóm người dùng: (i) quản trị viên (Admin) là cán bộ quản lý cơ sở vật chất, và (ii) người dùng (User) là giảng viên đặt và sử dụng phòng. Phạm vi áp dụng là các phòng học/phòng họp của một tòa nhà tám tầng, mỗi phòng có sức chứa, thiết bị và một trong hai loại khóa: khóa cơ/thẻ từ hoặc khóa mã số kết nối nền tảng OneIoT.

1.3 Định hướng giải pháp và tổ chức báo cáo

Nhóm lựa chọn React Native với TypeScript để xây dựng ứng dụng Android [1], dữ liệu được lưu cục bộ bằng Async Storage [2] để ứng dụng hoạt động độc lập trên một thiết bị. Song song, nhóm xây dựng dịch vụ Java Spring Boot [3] làm nền tảng đồng bộ dữ liệu, có thể bật bằng một cờ cấu hình. Với khóa mã số, ứng dụng kết nối trực tiếp tới OneIoT qua MQTT trên TLS [4] theo định dạng bản tin oneM2M [5] để gửi lệnh tạo mật khẩu tạm thời và nhận sự kiện mở khóa.

Phần còn lại của báo cáo được tổ chức như sau. Chương 2 trình bày tổng thể thành phần hệ thống, phạm vi MVP đã chốt và phân tích mã nguồn theo từng module. Chương 3 xây dựng quy trình kiểm thử, các kịch bản kiểm thử và kết quả. Phụ lục A liệt kê chi tiết thành phần của kho mã nguồn; Phụ lục B tổng hợp các prompt được sử dụng khi phát triển với trợ lý AI.

CHƯƠNG 2. TRIỂN KHAI HỆ THỐNG

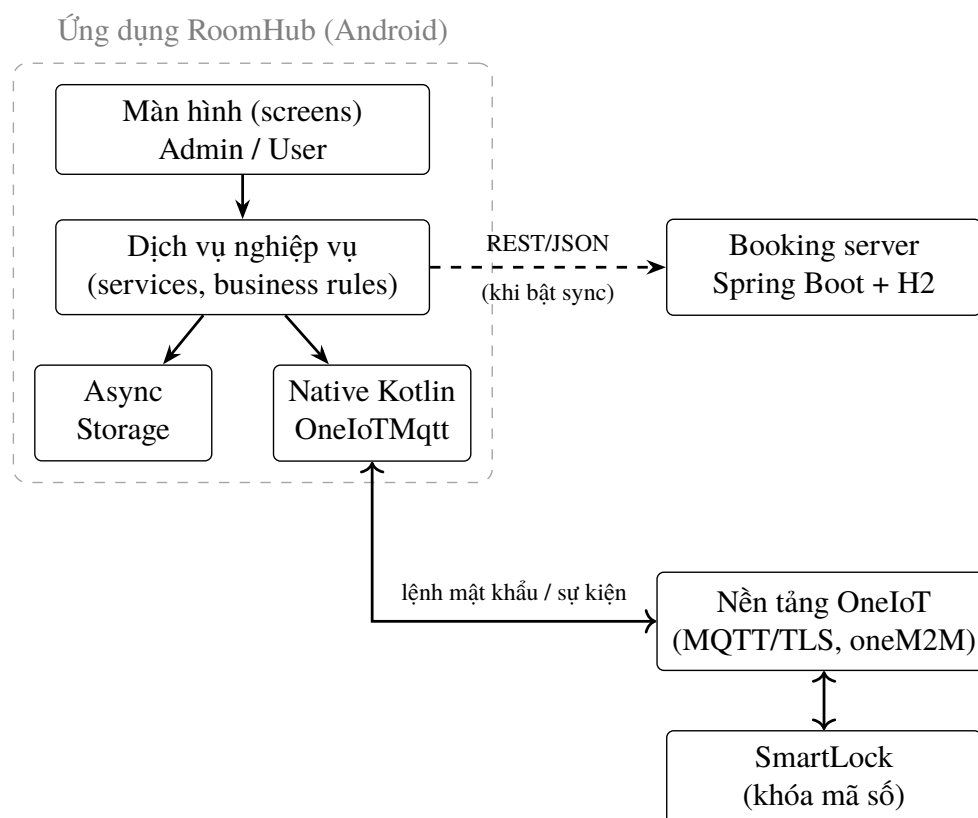
2.1 Tổng quan hệ thống

Hệ thống RoomHub gồm hai thành phần nằm chung trong kho mã nguồn `classroom-booking`, được tóm tắt ở Bảng 2.1. Thành phần trung tâm là ứng dụng di động; các thành phần còn lại phục vụ đồng bộ dữ liệu, giả lập thiết bị và lưu trữ tài liệu. Danh mục thư mục chi tiết được trình bày tại Phụ lục A.

Bảng 2.1: Các thành phần chính của hệ thống RoomHub

Thành phần	Công nghệ	Vai trò
Ứng dụng di động <code>code/booking_classroom</code>	React Native 0.86, TypeScript, Kotlin	Giao diện Admin/User, toàn bộ nghiệp vụ đặt phòng, lưu dữ liệu cục bộ, kết nối OneIoT qua module native.
Booking server <code>code/booking-server</code>	Java, Spring Boot, H2	REST API cho đăng nhập, phòng, booking, phê duyệt, mã tạm thời, thông báo; dùng khi bật đồng bộ.

Hình 2.1 mô tả kiến trúc tổng thể. Ứng dụng được tổ chức theo ba lớp: lớp màn hình (*screens*), lớp dịch vụ nghiệp vụ (*services*) và lớp lưu trữ.



Hình 2.1: Kiến trúc tổng thể của hệ thống RoomHub

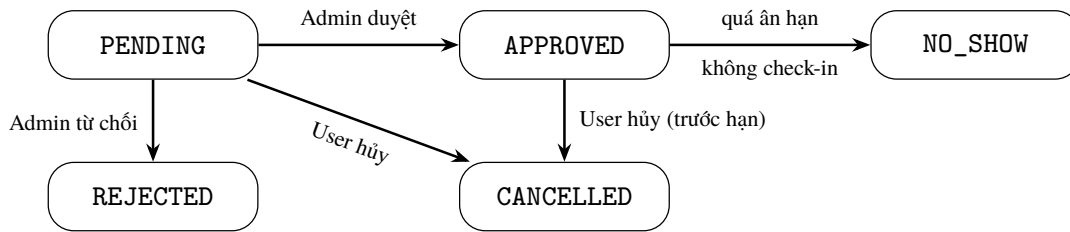
2.2 Phạm vi MVP

Tiêu chí hoàn thành MVP gồm: (i) luồng nghiệp vụ chính chạy được trên thiết bị thật, (ii) bản cài đặt Release tái lập được từ mã nguồn, (iii) các quy tắc nghiệp vụ quan trọng có kiểm thử tự động, và (iv) không còn lỗi nghiêm trọng đã biết trong luồng chính. Bảng 2.2 phân định rõ phần thuộc MVP và phần để lại cho giai đoạn sau.

Bảng 2.2: Phạm vi MVP đã chốt

Nhóm chức năng	Nội dung	Trạng thái
Tài khoản	Đăng nhập, đăng ký, đổi/khôi phục mật khẩu bằng mã khôi phục, hồ sơ, điều hướng theo vai trò; Admin quản lý tài khoản.	Đã triển khai
Tìm phòng	Lọc theo tầng, ngày giờ, sức chứa, thiết bị, loại khóa; sơ đồ tầng với màu trạng thái.	Đã triển khai
Đặt và duyệt	Tạo yêu cầu, kiểm tra quy tắc, Admin duyệt/từ chối/đổi phòng, hủy theo hạn.	Đã triển khai
Đặt lặp tuần	Tạo chuỗi tối đa 8 tuần, sửa/hủy các lượt tương lai, duyệt cả chuỗi.	Đã triển khai
Phòng và bảo trì	CRUD phòng, lịch bảo trì, yêu cầu bảo trì từ User, lịch sử sử dụng.	Đã triển khai
Truy cập phòng	Khóa cơ/thẻ từ: thỏa thuận lịch nhận khóa, ủy quyền nhận hộ. Khóa số: cấp quyền, tạo PIN 7 chữ số, gửi lệnh qua OneIoT.	Đã triển khai
Điểm danh	Check-in thủ công, ghi nhận no-show sau thời gian ân hạn.	MVP – xong
Thông báo, cấu hình	Thông báo theo sự kiện, banner ưu tiên; Admin cấu hình quy tắc.	Đã triển khai
Đồng bộ nhiều thiết bị	Dùng booking server làm nguồn dữ liệu chung.	Giai đoạn sau
Kiểm chứng khóa thật	Xác nhận SmartLock thật nhận và áp dụng mật khẩu.	Giai đoạn sau
Mở rộng	QR check-in, thống kê mức sử dụng, gợi ý phòng nâng cao.	Giai đoạn sau

Các quy tắc nghiệp vụ của MVP được tham số hóa trong kiểu `AppConfiguration` để Admin thay đổi ngay trong ứng dụng. Giá trị mặc định là: đặt trước tối thiểu 1 ngày và tối đa 3 ngày; mỗi User có tối đa 2 yêu cầu đang chờ hoặc sắp dùng; hạn hủy và hạn đổi phòng là 30 phút trước giờ bắt đầu; mã PIN có hiệu lực từ 10 phút trước đến 10 phút sau phiên; thời gian ân hạn trước khi ghi nhận no-show là 15 phút. Vòng đời trạng thái của một booking được thể hiện ở Hình 2.2.



Hình 2.2: Vòng đời trạng thái của một booking

2.3 Tổ chức mã nguồn ứng dụng

Mã nguồn ứng dụng được chia thành ba vùng. Vùng `src/core` chứa phần dùng chung ở mức hạ tầng: kiểu người dùng, hàm `readJson/writeJson` bọc Async Storage. Vùng `src/shared` chứa component giao diện dùng chung như `ScreenHeader` và `PasswordInput`. Vùng `src/modules` chứa mười module nghiệp vụ; mỗi module tuân theo cùng cấu trúc `screens/`, `components/`, `services/`, `model/` và chỉ công khai API qua `index.ts`. Quy ước này giúp mỗi thành viên phụ trách một module mà ít xung đột khi hợp nhất mã. Đoạn mã 2.1 minh họa cách một hàm dịch vụ hỗ trợ đồng thời hai chế độ dữ liệu.

Đoạn mã 2.1: Hàm dịch vụ chọn nguồn dữ liệu theo cờ cấu hình (rút gọn)

```

export async function createBooking(input: CreateBookingInput) {
  if (isRemoteApiEnabled()) { // chế độ đồng bộ qua
    ↪ server
    const r = await apiRequest<{booking:
      ↪ Booking}>('/api/bookings',
      {method: 'POST', body: input});
    return r.booking;
  }
  await assertAccountRole(input.requesterUsername, 'user');
  const bookings = await getBookings(); // chế độ local: Async
  ↪ Storage
  ...
}

```

2.4 Lập trình theo từng module

Bảng 2.3 tổng hợp mười module, khóa lưu trữ và người phụ trách chính. Các mục con sau đó phân tích những điểm lập trình quan trọng của từng module.

Bảng 2.3: Danh sách module của ứng dụng

Module	Phạm vi	Khóa lưu trữ
auth	Đăng nhập, đăng ký, đổi/khôi phục mật khẩu	auth.accounts
account_management	CRUD và phân quyền tài khoản (Admin)	dùng auth
room_management	CRUD phòng, tìm phòng, sơ đồ tầng	rooms.records
booking	Đặt, duyệt, hủy, lặp tuần, đổi phòng, điểm danh	booking.records
schedule_maintenance	Lịch và yêu cầu bảo trì	maintenance.*
access_control	Quyền tạo mã, PIN tạm thời	access.roomPermissions
smart_lock	Gắn khóa vào phòng, OneIoT, sự kiện khóa	smartLock.primary
notifications	Thông báo, banner ưu tiên	theo người dùng
profile	Hồ sơ cá nhân	theo người dùng
configuration	Tham số quy tắc nghiệp vụ	configuration.app

2.4.1 Module auth và account_management

Module auth lưu danh sách tài khoản kiểu AccountRecord gồm tên đăng nhập, mật khẩu, mã khôi phục, vai trò và trạng thái hoạt động. Lần chạy đầu, hệ thống khởi tạo hai tài khoản demo admin/1 và user/2. Hàm registerAccount kiểm tra tên đăng nhập 3–30 ký tự (chữ thường, số, dấu chấm, gạch ngang), mật khẩu và mã khôi phục tối thiểu 4 ký tự, và không trùng tên. Hàm resetPasswordWithRecoveryCode cho phép đặt lại mật khẩu khi tên đăng nhập và mã khôi phục khớp. Mọi thao tác ghi dữ liệu nhạy cảm ở các module khác đều gọi assertAccountRole để xác nhận tài khoản còn hoạt động và đúng vai trò, thay vì tin vào trạng thái giao diện. Phía account_management, hàm updateManagedAccount chặn Admin tự khóa hoặc tự đổi quyền của chính mình, còn deleteManagedAccount từ chối xóa tài khoản còn booking đang hoạt động.

2.4.2 Module room_management

Dữ liệu khởi tạo gồm 56 phòng, bảy phòng mỗi tầng trên tám tầng, với sức chứa và bộ thiết bị khác nhau. Các hàm createRoom, updateRoom, deleteRoom kiểm tra tầng trong khoảng 1–8, sức chứa nguyên dương, tên không trùng và không cho xóa phòng còn yêu cầu đang hoạt động. Hàm searchRooms lọc theo từ khóa, tầng, sức chứa tối thiểu, thiết bị và loại khóa. Màn hình RoomSearchScreen hiển thị kết quả trên sơ đồ tầng, tô màu theo trạng thái trống, đã đặt, bảo

trì hoặc không khớp bộ lọc. Màn hình RoomManagementScreen hợp nhất quản lý phòng và bảo trì: Admin chọn tầng, chọn phòng rồi sửa, xóa, lên lịch bảo trì hoặc xem lịch sử dụng.

2.4.3 Module booking

Đây là module lớn nhất (bookingRepository.ts, khoảng 760 dòng), tập trung toàn bộ quy tắc đặt phòng. Hàm validateBooking ở Đoạn mã 2.2 kiểm tra lần lượt: phòng tồn tại và sẵn sàng, định dạng ngày giờ, cửa sổ đặt trước, khoảng thời gian chưa kết thúc, không trùng bảo trì, không trùng booking khác của phòng, không trùng lịch của chính người đặt, giới hạn số yêu cầu đang hoạt động và mục đích sử dụng không rỗng. Hai khoảng thời gian được coi là trùng khi cùng ngày và `start < other.end` và `end > other.start`.

Đoạn mã 2.2: Kiểm tra quy tắc khi tạo booking (rút gọn)

```
if (!room || room.status !== 'AVAILABLE') throw new
  ↳ Error('Phòng không tồn tại...');
if (dayDiff < cfg.minAdvanceDays || dayDiff >
  ↳ cfg.maxAdvanceDays)
  throw new Error(`Chỉ được đặt trước từ ${cfg.minAdvanceDays}
    ↳ đến ...`);
if (await hasMaintenanceConflict(roomId, date, startTime,
  ↳ endTime))
  throw new Error('Phòng có lịch bảo trì trong khoảng thời gian
    ↳ này.');
```

```
const active = bookings.filter(b => isBookingStillActive(b));
if (active.some(b => b.roomId === roomId && overlaps(b, date,
  ↳ startTime, endTime)))
  throw new Error('Phòng đã có yêu cầu khác trong khoảng thời
    ↳ gian này.');
```

```
if (active.filter(b => b.requesterUsername === user).length >=
  ↳ cfg.maxActiveBookingsPerUser)
  throw new Error('Mỗi người chỉ có tối đa ... yêu cầu');
```

```
if (!input.purpose.trim()) throw new Error('Vui lòng nhập mục
  ↳ đích sử dụng phòng.');
```

Chức năng đặt lặp hàng tuần sinh tối đa 8 lượt cách nhau 7 ngày, gán chung recurringSeriesId và chỉ áp dụng cửa sổ đặt trước, giới hạn số yêu cầu cho lượt đầu tiên. Các thao tác trên chuỗi như reviewRecurringSeries, updatePendingRecurringBookings và cancelFutureRecurringBookings được viết theo nguyên tắc “tất cả hoặc không”: kiểm tra mọi lượt trước, chỉ ghi khi không lượt nào vi phạm. Khi duyệt, reviewBooking kiểm tra lại trùng lịch đã duyệt và bảo trì, rồi tự cấp quyền tạo mã nếu phòng dùng khóa số. Hàm changeBookingRoom chỉ cho đổi sang phòng thỏa meetsReplacementRoomRequirements: cùng loại khóa, sức chứa không nhỏ hơn và có đủ thiết bị của phòng cũ. Với điểm danh, confirmBookingNoShow chỉ chấp nhận

sau thời gian ân hạn và khi chưa có bằng chứng check-in, sau đó giải phóng khung giờ và thu hồi mã.

2.4.4 Module `schedule_maintenance`

Module quản lý hai loại dữ liệu: lịch bảo trì (`MaintenanceRecord`) do Admin tạo và yêu cầu bảo trì (`MaintenanceRequest`) do User gửi. Hàm `createMaintenance` từ chối lịch trùng bảo trì khác hoặc trùng booking đã duyệt, buộc Admin đổi phòng cho người dùng trước. Hàm `requestRoomMaintenance` chỉ cho phép User báo sự cố khi đang trong thời gian sử dụng một booking đã duyệt của chính mình, mỗi lượt sử dụng có tối đa một yêu cầu chờ xử lý. Phòng có yêu cầu đang chờ được tô vàng trên sơ đồ tầng của Admin.

2.4.5 Module `access_control`

Quyền tự tạo mã được lưu theo cặp User-phòng (`RoomPinPermission`), không theo từng booking, nên Admin có thể thu hồi quyền tại một phòng mà không ảnh hưởng phòng khác. Hàm `createTemporaryPin` kiểm tra booking đã duyệt, phòng dùng khóa số, chưa có mã còn hiệu lực, người tạo là chủ booking và còn quyền. Mã gồm 7 chữ số được sinh ngẫu nhiên, hiệu lực từ `start - pinGraceMinutes` đến `end + pinGraceMinutes`. Mã luôn được lưu cục bộ trước; nếu phiên `OneIoT` đang mở thì gửi ngay, nếu chưa thì `deliverPendingTemporaryPins` sẽ gửi các mã còn chờ khi kết nối (Đoạn mã 2.3). Cách làm này giúp người dùng vẫn nhận mã khi mất mạng.

Đoạn mã 2.3: Lưu mã trước, gửi tới khóa khi có kết nối (rút gọn)

```
const code = generateSevenDigitPin();
const validFrom = addMinutes(start, -cfg.pinGraceMinutes);
const validUntil = addMinutes(end, cfg.pinGraceMinutes);
const { lock, passwordId } = await
  ↪ reserveSmartLockPasswordId(booking.roomId);
let updated = await saveTemporaryPin(bookingId, { code,
  ↪ validFrom, validUntil,
                                lockPasswordId
                                ↪ Id:
                                ↪ passwordId
                                ↪ Id, ...
                                ↪ });
const status = await getOneIoTConnectionStatus().catch(() =>
  ↪ ({connected:false}));
if (status.connected) updated = await
  ↪ deliverTemporaryPin(updated, lock);
```

2.4.6 Module `smart_lock` và tích hợp `OneIoT`

Ứng dụng giao tiếp với SmartLock thông qua nền tảng `OneIoT` bằng giao thức MQTT bảo mật. Hệ thống cho phép gửi mã PIN tạm thời tới khóa và nhận các sự

kiện truy cập để cập nhật trạng thái booking. Thông tin xác thực OneIoT chỉ được lưu trong phiên làm việc và được xóa khi người dùng đăng xuất hoặc ứng dụng ngừng kết nối.

2.4.7 Module notifications, profile và configuration

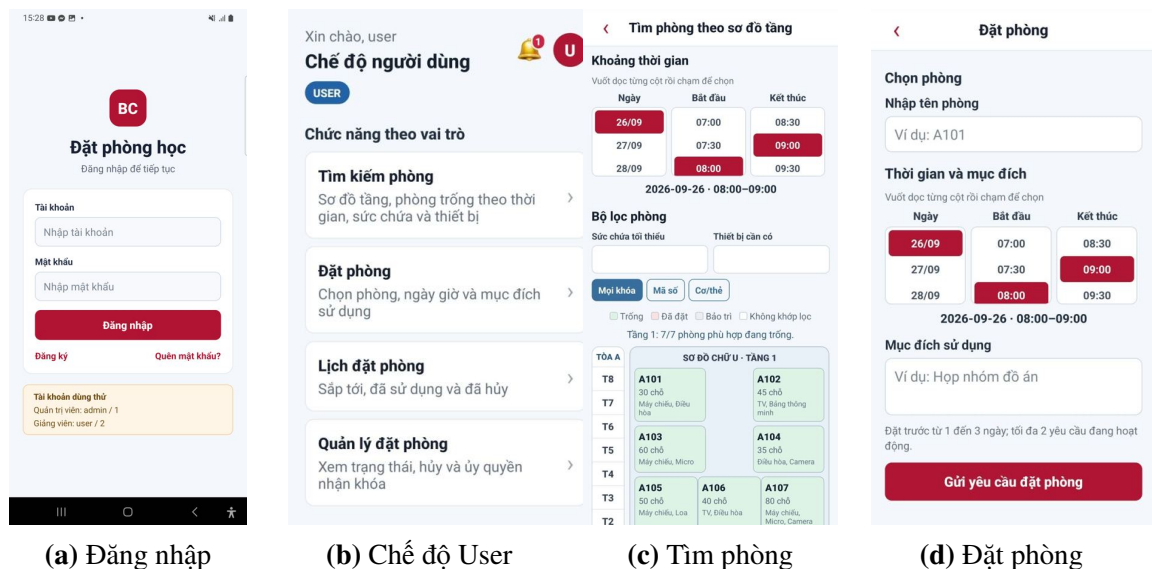
Hàm `addNotification` được các module nghiệp vụ gọi tại mỗi sự kiện như tạo yêu cầu, duyệt, từ chối, cấp mã, đổi phòng. Hàm `getNotificationTone` phân loại nội dung thành tích cực, tiêu cực, cần chú ý hoặc trung tính để tô màu. Thông báo quan trọng được đẩy thành banner hệ thống qua module native `PriorityNotificationPackage`. Module `profile` lưu họ tên, email, số điện thoại, đơn vị theo tài khoản. Module `configuration` kiểm tra các tham số là số nguyên không âm, số ngày tối thiểu không lớn hơn tối đa và giới hạn yêu cầu ít nhất là 1 trước khi lưu.

2.4.8 Booking server

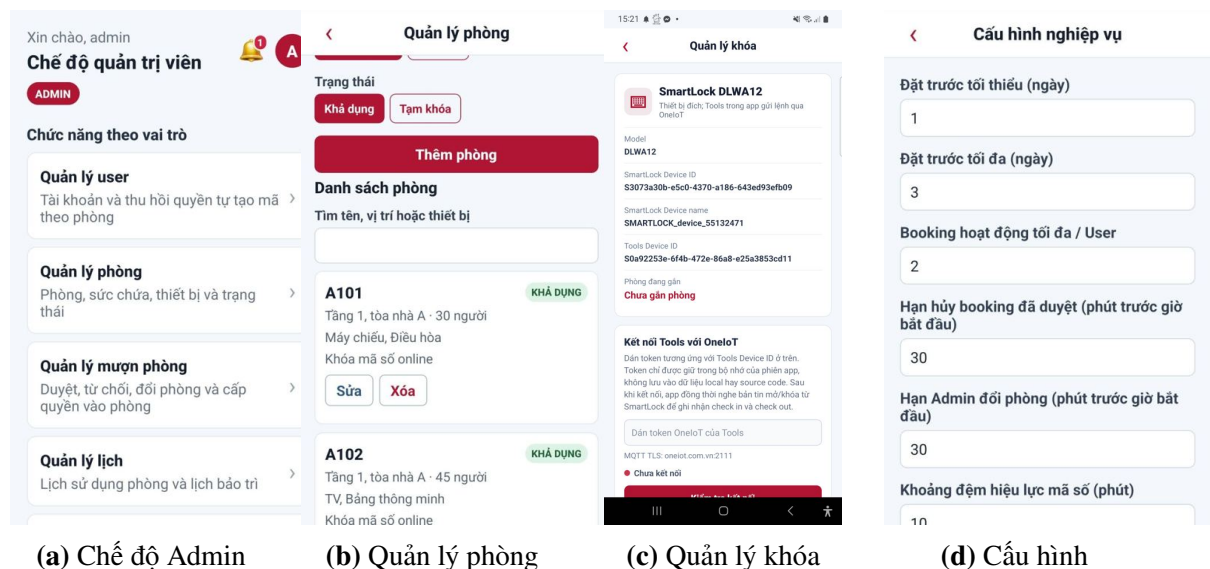
Booking server gồm bảy lớp Java: `ApiController` khai báo các endpoint `/auth/login`, `/rooms`, `/configuration`, `/maintenance`, `/bookings`, `/bookings/{id}/review`, `/bookings/{id}/temporary-pin`, `/pin-permissions`, `/notifications`; `BookingService` cài đặt lại quy tắc nghiệp vụ phía server; `SessionAuthFilter` kiểm tra token Bearer; `ApiExceptionHandler` chuẩn hóa thông báo lỗi; `DataSeeder` tạo dữ liệu mẫu. Mặc định server dùng H2 dạng tệp cục bộ, profile `remote` kết nối H2 từ xa qua SSL.

2.5 Kết quả giao diện

Hình 2.3 và Hình 2.4 là ảnh chụp ứng dụng bản Release chạy trên điện thoại Samsung M21. Sau khi đăng nhập, ứng dụng tự mở chế độ theo vai trò; User tìm phòng theo khung giờ và bộ lọc trên sơ đồ tầng rồi gửi yêu cầu, Admin quản lý phòng, quyền tạo mã, khóa thông minh và cấu hình quy tắc.



Hình 2.3: Giao diện phía người dùng



Hình 2.4: Giao diện phía quản trị viên

2.6 Phân công và đóng góp

Bảng 2.4 tổng hợp phân công công việc và đóng góp chính của các thành viên trong nhóm.

Bảng 2.4: Phân công công việc trong nhóm

Thành viên	Phần việc chính
Đỗ Thùy Linh	Xây dựng dịch vụ Booking Server bằng Spring Boot; cấu hình cơ sở dữ liệu H2 từ xa; phát triển banner thông báo ưu tiên; thiết kế logo; xây dựng chức năng ẩn/hiện mật khẩu.
Nguyễn Đình Phúc	Phát triển chức năng đặt phòng lặp hàng tuần và quản lý chuỗi đặt phòng; xây dựng cơ chế thông báo phê duyệt; quản lý trường hợp no-show; xây dựng kiểm thử tự động cho chức năng booking.
Nguyễn Việt Dũng	Xây dựng quy trình và các kịch bản kiểm thử; thực hiện kiểm thử công toàn bộ các luồng chức năng; tổng hợp và phân loại 14 nhóm lỗi của hệ thống (Chương 3). Xây dựng hệ thống nghiệp vụ và tài liệu dự án.
Hoàng Văn Tùng	Xây dựng khung ứng dụng và kiến trúc module; phát triển phần lớn giao diện và nghiệp vụ; tích hợp OneIoT và chức năng PIN tạm thời; xây dựng mô phỏng SmartLock và gateway; thực hiện build Release.

CHƯƠNG 3. QUY TRÌNH VÀ KỊCH BẢN KIỂM THỬ

3.1 Mục tiêu và phương pháp kiểm thử

Ứng dụng được kiểm thử bằng các **kịch bản đầu cuối trên điện thoại thật**: cài bản Release từ APK, thao tác trực tiếp trên màn hình ở cả hai vai trò Admin và User, rồi đối chiếu kết quả với quy tắc nghiệp vụ [6].

Kịch bản bao gồm luồng nghiệp vụ chính, các giá trị biên (cửa sổ đặt 1–3 ngày, sức chứa, giới hạn yêu cầu, thời gian ân hạn) và chuyển trạng thái booking (Hình 2.2). Chỉ bản build đạt bộ Jest [7] mới được đưa lên thiết bị.

3.2 Môi trường kiểm thử trên thiết bị

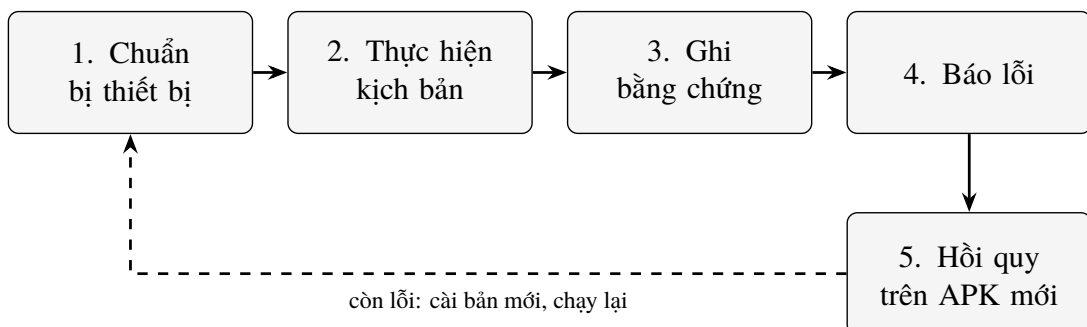
Các kịch bản nhiều vai trò được thực hiện bằng cách đăng nhập luân phiên Admin và User trên cùng một máy; các quy tắc theo thời gian được kiểm tra bằng cách chỉnh ngày giờ thiết bị.

Bảng 3.1: Môi trường kiểm thử trên thiết bị thực tế

Hạng mục	Cấu hình
Thiết bị	Samsung Galaxy M21 (SM-M215F), Android
Bản cài đặt	app-release.apk build từ mã nguồn, cài bằng <code>adb install -r</code>
Công cụ theo dõi	<code>adb logcat</code> lọc lỗi <code>AndroidRuntime</code> , <code>ReactNativeJS</code> ; ảnh chụp màn hình làm bằng chứng
Dữ liệu ban đầu	Xóa dữ liệu ứng dụng để về 56 phòng mẫu (8 tầng $\times$ 7 phòng), tài khoản admin/1, user/2
Cấu hình mặc định	Đặt trước 1–3 ngày, tối đa 2 yêu cầu, hủy trước 30 phút, ân hạn PIN 10 phút, no-show 15 phút
Mạng	Wi-Fi bật/tắt để kiểm thử trường hợp mất kết nối; OneIoT <code>oneiot.com.vn:2111</code> (MQTT/TLS)

3.3 Quy trình kiểm thử trên thiết bị

Quy trình gồm năm bước, lặp lại sau mỗi bản APK sửa lỗi, như Hình 3.1.



Hình 3.1: Quy trình kiểm thử trên thiết bị thực tế

Mỗi lỗi được ghi kèm kịch bản liên quan, các bước tái hiện, ảnh chụp màn hình và mức độ: *Nghiêm trọng* (sai dữ liệu, chặn luồng chính), *Cao* (sai quy tắc nghiệp vụ), *Trung bình* (thiếu kiểm tra đầu vào) và *Thấp* (giao diện). Lỗi chỉ được đóng khi kịch bản đạt trên bản APK mới.

3.4 Kịch bản kiểm thử trên thiết bị

Bảng 3.2 trình bày mười hai kịch bản; cột “Lỗi” tham chiếu tới Bảng 3.4.

Bảng 3.2: Kịch bản kiểm thử đầu cuối trên thiết bị thực tế

Mã	Các bước trên thiết bị	Kết quả mong đợi	Lỗi
KB-01	Cài đặt và khởi động. (1) Cài APK bằng <code>adb install -r</code> ; (2) rút cáp, buộc dừng ứng dụng; (3) mở lại từ màn hình chính.	Ứng dụng mở màn hình đăng nhập, không cần Metro; logcat không có lỗi crash.	–
KB-02	Đăng nhập theo vai trò. (1) Đăng nhập <code>admin/1</code> ; (2) đăng xuất, đăng nhập <code>user/2</code> ; (3) nhập sai mật khẩu.	(1) Chế độ quản trị viên; (2) chế độ người dùng; (3) báo sai thông tin. Màn hình không có chữ thừa.	L03
KB-03	Đăng ký và khôi phục mật khẩu. (1) Chọn Đăng ký, bỏ trống mã khôi phục; (2) nhập đủ, tạo <code>teacher01</code> ; (3) đăng ký lại tên <code>teacher01</code> ; (4) Quên mật khẩu, nhập mã khôi phục, đặt mật khẩu mới; (5) đăng nhập bằng mật khẩu mới.	(1) Cảnh báo trường bắt buộc có dấu (*); (2) tạo thành công; (3) báo trùng tên; (4)–(5) đăng nhập được, mật khẩu cũ bị từ chối.	L02 L03 L04
KB-04	Hồ sơ cá nhân. (1) User mở Thông tin cá nhân, sửa họ tên, email, số điện thoại, nhấn Lưu; (2) đóng hẳn ứng dụng rồi mở lại; (3) đăng nhập Admin, mở Quản lý user xem tài khoản đó.	Thông tin được giữ sau khi mở lại; Admin thấy cùng thông tin; User không thay đổi được vai trò của mình.	L05 L14
KB-05	Tìm phòng trên sơ đồ tầng. (1) User mở Tìm phòng, chọn ngày mai 08:00–09:00; (2) nhập sức chứa tối thiểu 45; (3) chọn thiết bị Máy chiếu; (4) lần lượt chọn tầng T1–T8.	Mọi phòng ≥ 45 chỗ có máy chiếu được tô xanh, kể cả phòng đúng 45 chỗ; phòng không khớp được làm mờ.	L06 L07

Mã	Các bước trên thiết bị	Kết quả mong đợi	Lỗi
KB-06	Đặt phòng và phê duyệt. (1) User chọn phòng A102, ngày mai 09:00–11:00, bỏ trống Mục đích sử dụng, nhấn Gửi; (2) nhập “Hợp môn đồ án”, gửi lại; (3) đăng nhập Admin, mở Quản lý mượn phòng, duyệt; (4) đăng nhập User, mở Lịch đặt.	(1) Cảnh báo tại trường Mục đích; (2) yêu cầu ở trạng thái Chờ duyệt; (3) Admin thấy huy hiệu và thông báo mới; (4) User thấy Đã duyệt và thông báo màu xanh.	L09 L11 L12
KB-07	Quy tắc đặt phòng. (1) User đặt phòng cho hôm nay và cho 4 ngày sau; (2) tạo yêu cầu thứ ba khi đã có hai; (3) đăng nhập teacher01, đặt A102 cùng khung giờ ở KB-06; (4) Admin mở Cấu hình nghiệp vụ, đổi tối đa thành 3; User đặt lại bước (2).	(1)–(3) bị từ chối với thông báo đúng quy tắc; (4) sau khi đổi cấu hình, yêu cầu thứ ba được tạo.	–
KB-08	Đặt lặp hàng tuần. (1) User bật Lặp hàng tuần, 4 tuần, gửi yêu cầu; (2) mở Lịch đặt xem chuỗi; (3) Admin duyệt cả chuỗi; (4) User hủy các lượt tương lai.	Hiển thị đủ 4 lượt với ngày, giờ và trạng thái từng tuần; duyệt và hủy áp dụng cho cả chuỗi, đồng bộ giữa hai vai trò.	L10 L14
KB-09	Quản lý phòng và bảo trì. (1) Admin mở Quản lý phòng, thêm phòng mới ở tầng 7; (2) lên lịch bảo trì một phòng vào ngày mai; (3) User tìm phòng ngày mai; (4) Admin xóa phòng đang có booking đã duyệt.	(1) Phòng mới xuất hiện trên sơ đồ tầng 7; (3) phòng bảo trì màu xám, không đặt được; (4) từ chối xóa.	L08 L13
KB-10	Báo sự cố khi đang dùng phòng. (1) Chỉnh giờ thiết bị vào giữa phiên đã duyệt của User; (2) User mở booking, chọn Báo sự cố, nhập lý do; (3) Admin mở Quản lý phòng.	Admin nhận thông báo; tầng và phòng liên quan tô vàng; Admin tiếp nhận và lên lịch bảo trì được.	–

Mã	Các bước trên thiết bị	Kết quả mong đợi	Lỗi
KB-11	Khóa cơ/thẻ từ và điểm danh. (1) Với booking đã duyệt ở phòng khóa cơ, User đề xuất giờ nhận khóa; (2) Admin đề xuất giờ và địa điểm khác; (3) User chấp nhận; (4) chỉnh giờ vào phiên, Admin xác nhận có mặt; (5) với booking khác không check-in, chỉnh giờ qua 15 phút sau giờ bắt đầu, Admin ghi vắng mặt.	(3) Hai bên thấy lịch đã thống nhất; (4) booking có check-in; (5) trước 15 phút bị từ chối, sau 15 phút chuyển Vắng mặt và khung giờ được giải phóng.	–
KB-12	Khóa số và OneIoT. (1) Admin cho phép đặt cùng ngày, gắn SmartLock vào A101; (2) User đặt A101 hôm nay khi phiên đã bắt đầu, Admin duyệt; (3) tắt Wi-Fi, User tạo mã tạm thời; (4) bật Wi-Fi, Admin nhập token Tools, nhấn Kiểm tra kết nối; (5) chuyển ứng dụng xuống nền.	(2) Duyệt được vì phiên chưa kết thúc; (3) mã 7 chữ số hiện ngay, trạng thái chờ gửi; (4) kết nối thành công, lệnh tự gửi; (5) phiên OneIoT tự ngắt, token không còn lưu.	–

3.5 Kết quả kiểm thử và lỗi đã phát hiện

Bảng 3.3 tổng hợp kết quả trên Samsung M21.

Bảng 3.3: Kết quả thực hiện kịch bản trên thiết bị

Mã	Kết quả	Ghi chú
KB-01	Đạt	Cài đặt, khởi động lại không cần Metro; 0 lỗi crash trong logcat
KB-02	Đạt, có góp ý	Điều hướng đúng vai trò; còn chữ thừa trên màn hình đăng nhập (L03)
KB-03	Không đạt	Thiếu đánh dấu bắt buộc, mã khôi phục chưa bắt buộc, lỗi tạo tài khoản (L02–L04)
KB-04	Không đạt	Chưa kiểm tra quyền khi sửa hồ sơ, thông tin không đồng bộ (L05, L14)
KB-05	Không đạt	Lọc sức chứa 45 chỉ hiện phòng 40 chỗ; chưa chọn từng thiết bị (L06, L07)
KB-06	Không đạt	Thiếu cảnh báo Mục đích; lỗi đặt phòng phía server và lỗi khi duyệt (L09, L11, L12)
KB-07	Đạt	Các quy tắc cửa sổ đặt, trùng lịch, giới hạn yêu cầu hoạt động đúng
KB-08	Không đạt	Lịch sử chỉ hiện ngày đầu của chuỗi; đặt lặp không đồng bộ (L10, L14)
KB-09	Không đạt	Không thêm được phòng; trạng thái đặt giờ không đồng bộ; nút Xóa thiếu lọc (L08, L13)
KB-10	Đạt	Báo sự cố và tô vàng phòng hoạt động đúng
KB-11	Đạt	Thỏa thuận nhận khóa, check-in, no-show đúng ân hạn
KB-12	Đạt một phần	Lệnh đã gửi tới OneIoT; chưa kiểm chứng khóa thật mở bằng mã

Các lỗi phát hiện được tổng hợp ở Bảng 3.4.

Bảng 3.4: Danh sách lỗi và vấn đề đã báo cho nhóm

Mã	Module	Mô tả	Mức độ
L01	Giao diện	Cần sửa logo và màu sắc	Thấp
L02	auth, profile	Thiếu Email, Mã nhân viên; trường bắt buộc chưa có dấu (*)	Trung bình
L03	auth	Cập nhật Database; chữ thừa trên màn hình đăng nhập; mã khôi phục phải bắt buộc	Trung bình
L04	auth	Xung đột khi đăng ký; lỗi tạo tài khoản liên quan Database	Nghiêm trọng
L05	profile	Cập nhật thông tin cá nhân chưa kiểm tra Role/Quyền	Cao
L06	Phòng	Thiết bị chưa chọn được từng mục	Trung bình
L07	Phòng	Tìm theo sức chứa bỏ qua giới hạn: chọn 45 chỉ hiển thị 40	Cao
L08	Phòng	Nút Xóa cần bộ lọc tự động	Thấp

Mã	Module	Mô tả	Mức độ
L09	booking	“Mục đích sử dụng” chưa cảnh báo khi bỏ trống	Trung bình
L10	booking	Lịch sử đặt theo tuần chỉ hiển thị ngày đầu tiên	Cao
L11	booking	Bug liên quan server khi đặt phòng	Nghiêm trọng
L12	booking	Lỗi trong quá trình duyệt	Cao
L13	Phòng, bảo trì	Không thêm được phòng; trạng thái “Đặt giờ phòng không đồng bộ”	Nghiêm trọng
L14	Đồng bộ dữ liệu	Đặt phòng và phê duyệt, thông tin cá nhân và tài khoản, đặt lịch hàng tuần không đồng bộ giữa các chức năng	Nghiêm trọng

Bốn lỗi Nghiêm trọng (L04, L11, L13, L14) đều liên quan đến đồng bộ dữ liệu và cần được ưu tiên xử lý, tiếp đến là các lỗi sai quy tắc nghiệp vụ, cuối cùng là giao diện.

KẾT LUẬN

Nhóm đã hoàn thành MVP của RoomHub: một ứng dụng Android chạy độc lập, bao phủ đầy đủ vòng đời mượn phòng từ tìm phòng trên sơ đồ tầng, đặt đơn lẻ hoặc lập tuần, phê duyệt và đổi phòng, đến cấp quyền truy cập bằng khóa cơ/thẻ từ hoặc mật khẩu tạm thời. Mã nguồn được tổ chức thành mười module có ranh giới rõ, quy tắc nghiệp vụ tập trung ở lớp dịch vụ và được kiểm thử tự động. Quy trình kiểm thử năm bước cùng 12 kịch bản đầu cuối trên thiết bị thực tế đã phát hiện 14 nhóm vấn đề, trong đó lỗi đồng bộ dữ liệu là trọng tâm cần xử lý.

Hướng phát triển tiếp theo gồm: (i) bật đồng bộ qua booking server để dùng chung dữ liệu trên nhiều thiết bị và khắc phục nhóm lỗi đồng bộ; (ii) kiểm chứng đầu cuối với SmartLock thật; (iii) bổ sung QR check-in/check-out, thống kê mức sử dụng phòng và gợi ý phòng nâng cao.

TÀI LIỆU THAM KHẢO

- [1] Meta Platforms, “React Native – Learn once, write anywhere.” <https://reactnative.dev/docs/getting-started>. Truy cập 10/2026.
- [2] React Native Community, “Async Storage – An asynchronous, persistent, key-value storage system for React Native.” <https://react-native-async-storage.github.io/async-storage/>. Truy cập 10/2026.
- [3] VMware Tanzu, “Spring Boot Reference Documentation.” <https://docs.spring.io/spring-boot/>. Truy cập 10/2026.
- [4] OASIS, “MQTT Version 3.1.1, OASIS Standard,” 2014. <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html>.
- [5] oneM2M Partners, “oneM2M TS-0010: MQTT Protocol Binding.” <https://www.onem2m.org/technical/published-specifications>. Truy cập 10/2026.
- [6] ISTQB, “Certified Tester Foundation Level Syllabus v4.0,” 2023. <https://www.istqb.org/>.
- [7] OpenJS Foundation, “Jest – Delightful JavaScript Testing.” <https://jestjs.io/docs/getting-started>. Truy cập 10/2026.

PHỤ LỤC

A. TỔNG THỂ THÀNH PHẦN DỰ ÁN

A.1 Cấu trúc kho mã nguồn

Kho mã nguồn <https://github.com/xwomen1/classroom-booking> được tổ chức như Đoạn mã A.1. Kho chỉ lưu mã nguồn cần thiết; tệp APK, bộ nhớ đệm, node_modules và thư mục build không được đưa lên Git, mỗi thành viên tự build sau khi clone.

Đoạn mã A.1: Cấu trúc thư mục chính

```
classroom-booking/
|-- README.md                      Hướng dẫn clone, build, cài
  ↳ APK
|-- du_an_dat_phong.tex/.pdf       Đề xuất dự án
|-- workflow_booking_app.tex/.pdf Workflow nghiệp vụ
|-- hien_trang_trien_khai.tex/.pdf Hiện trạng triển khai
|-- code/
|   |-- booking_classroom/        Ứng dụng React Native
|   ↳ (Android)
|       |-- App.tsx               Điểm vào, điều hướng theo vai
|       ↳ trò
|       |-- src/core/             api client, cò cấu hình,
|       ↳ storage
|       |-- src/shared/           component dùng chung
|       |-- src/modules/         10 module nghiệp vụ
|       |-- __tests__/           36 ca kiểm thử Jest
|       |-- android/.../OneIoTMqttPackage.kt  MQTT/TLS tới
|       ↳ OneIoT
|       |-- android/.../PriorityNotificationPackage.kt  banner
|       |-- docs/, evidence/     sơ đồ luồng, ảnh kiểm tra
|       ↳ thiết bị
|       |-- `-- scripts/         build Debug/Release
|       ↳ (PowerShell)
|   |-- `-- booking-server/      Spring Boot + H2
|       |-- `-- src/main/java/com/bookingclassroom/server/
|       |       ApiController, BookingService, SessionAuthFilter,
|       |       ApiExceptionHandler, DataSeeder, Domain
|-- tunghv2/                      Giả lập SmartLock (Python)
`-- tunghv3/                      Gateway HTTP-MQTT cũ (Python)
```

A.2 Build và cài đặt

Ứng dụng được build và cài đặt bằng các lệnh trong Đoạn mã A.2. Bản Release chạy độc lập sau khi cài, không cần Metro, cáp USB hay booking server.

Đoạn mã A.2: Build và cài đặt bản Release

```
git clone https://github.com/xwomen1/classroom-booking.git
cd classroom-booking/code/booking_classroom
npm ci
npm run build:android:release
adb install -r
  ↳ ./android/app/build/outputs/apk/release/app-release.apk
npm test      # chạy bộ kiểm thử Jest
npm run lint   # kiểm tra ESLint
```

A.3 Tổng hợp tài liệu và phụ lục

Bảng A.1 tóm tắt các tài liệu đi kèm dự án và nơi chúng được tham chiếu trong báo cáo.

Bảng A.1: Tài liệu đi kèm dự án

Tài liệu	Nội dung
du_an_dat_phong.pdf	Đề xuất dự án, phạm vi, tiêu chí kiểm chứng
workflow_booking_app.pdf	Workflow nghiệp vụ đặt phòng, phê duyệt, bàn giao
hien_trang_trien_khai.pdf	Hiện trạng các chức năng đã kiểm tra
evidence/device_verification.txt	Nhật ký kiểm tra trên Samsung M21
RoomHub_final.pptx	Slide trình bày sản phẩm
Phụ lục B	Prompt sử dụng trong quá trình phát triển

B. PROMPT SỬ DỤNG TRONG QUÁ TRÌNH PHÁT TRIỂN

Nhóm sử dụng trợ lý AI lập trình như một công cụ hỗ trợ trong phân tích yêu cầu, sinh mã, viết kiểm thử và soạn tài liệu. Mọi đầu ra của AI đều được thành viên phụ trách đọc lại, chạy kiểm thử và chỉnh sửa trước khi commit. Phụ lục này tổng hợp các prompt mẫu theo từng giai đoạn và các quy tắc kiểm soát AI được đặt trong kho mã nguồn.

B.1 Nguyên tắc viết prompt

Các prompt của nhóm tuân theo bốn nguyên tắc: (i) cung cấp đủ ngữ cảnh gồm module, tệp liên quan và quy tắc nghiệp vụ; (ii) giới hạn phạm vi sửa trong một module để tránh thay đổi lan rộng; (iii) yêu cầu kèm kiểm thử và lệnh kiểm tra (`npm test`, `npm run lint`, `tsc`); và (iv) yêu cầu AI không đánh dấu chức năng là hoàn thành khi chưa có mã và kiểm thử tương ứng, đúng với quy ước trong `src/modules/README.md`.